Mayalok: Round 2 Technical Note (POIs and Navigation)

Techfest 2026-27, IIT Bombay | Round 2: POIs & Navigation | Team QWERTY | Team ID: TM-E434D6B6C8 | Competition ID: NAMM-0B0BD6B7353A

This note describes only what is implemented in the repository (`web/src` , `pipeline/namma/navgrid.py`). Items we could not confirm from code are marked **(unknown)**.

1. POI schema, zones and admin mode

Each scene folder holds a `pois.json` (`{ "version": 1, "pois": [...] }`). A POI is:

Field	Type	Meaning
`id`	string	Stable key, also usable in deep links (`?to=<id>`)
`name`	string	Label shown in 3D and in search
`category`	string	One of `room, furniture, electronics, facility, exit, shop, food, info, other`; unknown values fall back to `other` (icon and colour)
`tags`	string[] (optional)	Extra search keywords, e.g. `couch`, `3-seater`
`description`	string (optional)	Free text, lowest search weight
`position`	[x, y, z] metres, +Y up	Label anchor in the scene's metric frame
`zone`	[[x, z], ...] (optional)	Floor polygon, 3+ vertices, for rooms and areas

Spatial indexing. All coordinates share the metric, floor-at-zero frame produced by the pipeline, the same frame as the navgrid. Zone POIs (rooms) are drawn as a tinted floor polygon with an outline; point POIs (objects) get a label with a stem down to the floor. `zoneAt(point)` returns the zone containing a floor point (smallest area wins, so a counter zone inside a kitchen zone is found first); `nearestPoint` finds the closest point POI within 0.9 m of a clicked surface. Labels fade beyond 14 m and are dimmed when the navgrid says a wall blocks the line of sight (refresh about 8 Hz).

Interaction. Click a 3D label to select it; the selection panel offers "Go there" (teleport to a viewpoint facing the POI, key `T`) and "Navigate" (key `G`). The minimap shows POIs too, and clicking an empty spot teleports to the nearest walkable cell.

Admin mode. Open the viewer with `?admin` (or press the pencil button) to edit places: "+ Add place", then click a spot in the 3D view (label height comes from the hit point, clamped 0.7 to 2.2 m); fill name, category, comma-separated keywords, description and label height. "Draw zone" then lets you click corners on the minimap and "Finish zone" (needs 3+ points). Edits are kept in the browser's `localStorage` (key `namma.pois.<sceneId>` ; key prefix still carries the old project name) and are applied on reload. "Export" downloads `pois.json` to place into the scene folder; "Import" loads one (validated only for a `pois` array); "Reset" discards local edits. There is no server-side save by design: the `pois.json` in the scene folder is the source of truth.

Shipped data. `demo-apartment` (synthetic test scene) has 10 POIs (3 rooms with zones, 7 objects). `incubation` (real capture, partial) currently has an **empty** `pois.json`; its POIs are still to be annotated.

2. Search (Fuse.js)

`lib/search.ts` builds a Fuse.js index over the POI list:

- Keys and weights: `name` 0.6, `tags` 0.25, `category` 0.1, `description` 0.05.
- `threshold: 0.4` (typo tolerant), `ignoreLocation: true` (matches anywhere in the string).
- Result limit 8 in the UI; an empty query returns the first POIs unfiltered.

UI (`SearchBar`): press `/` anywhere to focus (this releases pointer lock); arrow keys move, Enter selects and jumps, Shift+Enter or the "Go" button starts navigation; each result shows its category and live distance from the player. The index is rebuilt when the POI list changes (including admin edits). The deep link `?to=<id or text>` resolves by exact id, else the top Fuse match, and starts navigation once the scene has loaded.

3. Pathfinding (A*)

Grid. The pipeline (`namma.navgrid`) slices the cleaned splat into a 2D occupancy grid; defaults are resolution **0.05 m (5 cm)**, agent radius **0.25 m**, obstacle height band **0.12 m to 1.9 m**. Floor cells come from splats near y = 0 (|y| < 0.08 m); cells where the phone walked are forced free. A cell is walkable if its distance to the nearest blocked cell (Euclidean distance transform) is at least the agent radius, i.e. **obstacles are inflated by 0.25 m**, or it lies on the walked path. Only the largest connected walkable component is kept. A per-cell clearance map (cm, saturating at 255) is exported alongside. Shipped grid sizes: demo apartment 190 x 150 cells, incubation scene 95 x 140 cells, both at 5 cm. The grid is serialised in `navgrid.json` as base64 uint8 row-major arrays.

Search. `lib/astar.ts`:

- 8-connected; diagonal cost sqrt(2); **no corner cutting** (a diagonal step requires both adjacent orthogonal cells to be walkable).
- Heuristic: octile distance `dr + dc + (sqrt2 - 2) * min(dr, dc)` in cells. The wall penalty only raises step costs, so the heuristic never overestimates.
- Wall-aware cost: each step costs `base * (1 + wallPenalty * max(0, 1 - clearance / preferredClearance))` with defaults `preferredClearance = 0.6 m`, `wallPenalty = 2.0`. Paths therefore centre themselves in corridors and doorways, paying up to 3x distance when hugging a wall.
- Binary min-heap open set, typed arrays for scores. Start and goal snap to the nearest walkable cell (ring search, up to 50 m).
- For a zone POI the goal is the zone centroid; for a point POI its floor position.

Smoothing. String-pulling over the cell path: points are dropped while the straight segment between anchor and candidate stays on walkable cells (sampled every half cell). The route starts exactly at the user's position when that spot is walkable.

Web Worker. `Router` posts the grid to `pathWorker.ts` once (`init`), then sends `plan` requests (start, goal, id) and resolves promises on reply, so planning never blocks rendering. If a Worker cannot be created it plans inline. Planning time (ms) is returned with each route and logged to the console; we have no benchmark table for it **(unknown)**.

Re-routing. While following a route, if the player drifts more than 1.6 m from the polyline (at least 1.5 s since the last plan, and not during auto-walk), the route is re-planned from the current position.

4. Path rendering and turn-by-turn

Rendering (`PathLayer`). The smoothed points become a centripetal Catmull-Rom curve, resampled every 8 cm into a 0.5 m wide ribbon lying 4 cm above the floor. A shader scrolls chevrons toward the goal, fades the ends, and dims ribbon sections whose line of sight from the camera is blocked in the navgrid (splats do not write depth, so this stands in for occlusion; checked at about 8 Hz, sections beyond 30 m are dimmed). A pulsing ring and light pillar mark the destination. The route is also drawn on the minimap.

Turn-by-turn (`buildInstructions`, `format.ts`). At each interior route vertex the signed turn angle is classified: under 20 degrees is straight (no instruction), 20 to 50 slight left/right, 50 to 135 left/right, over 135 u-turn. The list is `start`, the turns with their distance along the route, and `arrive`. `NavHud` shows an arrow, text such as "Turn left in 6 m" ("now" within 1.5 m), the destination name, remaining distance and ETA at 1.2 m/s. Progress is the projection of the player onto the route; arrival triggers within 0.9 m of the end. "Auto-walk" (`F`) drives the camera along the route; `X` ends navigation.

5. Limitations

- The grid is 2D: single floor only; stairs, ramps and multi-level venues are not modelled.
- Obstacles come from splat density, so thin or transparent objects, glass and mirrors can be missed or ghosted; unreconstructed regions are not walkable unless the capture walked through them. The incubation scene is a partial reconstruction (65 of 293 keyframes registered).
- Scale for the incubation scene comes from the 1.35 m phone-height assumption, not ArUco markers; distances and ETA are only as accurate as that scale.
- The 0.25 m agent radius is fixed at grid build time; there is no UI profile for a wider agent **(unknown whether any runtime override beyond `PlanOptions` is wired up)**.
- Search is lexical (Fuse.js); there are no synonyms or semantic matching beyond the tags authors add.
- Admin edits are browser-local until exported and copied into the scene folder; there is no multi-user editing.
- Dynamic obstacles (people, moved furniture) are not detected.
- The repo includes `astar.test.ts` (vitest) for routing; we do not quote coverage figures.

6. Adding a venue

1. Capture with the SOP (`docs/SOP.md`), then run `namma build --input <folder> --work <workdir> --gsplat-dir <gsplat> --out <scene> --name "<Site>"` (or `namma process-splat` for an exported `.ply`).
2. Copy the output folder (`scene.spz`, `scene.json`, `navgrid.json`, `minimap.png`, `pois.json`, `report.json`) to `web/public/scenes/<id>/`.
3. Add `{ "id", "name", "description" }` to `web/public/scenes/index.json`; the first entry is the default, others open via `?scene=<id>`.
4. Open `?scene=<id>&admin`, add POIs and draw zones, then Export and replace the scene's `pois.json`.
5. Rebuild and deploy the static site (`npm run build` in `web/`).